



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных
технологий

Отчет по итоговому проекту
«WEB-приложение для сохранения и отображения изображений»

по дисциплине «Технологии разработки программных приложений»

Выполнил:

Студент группы ИКБО-20-23

Кузнецов Л.А.

Проверил:

Доцент кафедры МОСИТ, кандидат
технических наук, доцент

Чернов Е.А.

Москва 2025 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	4
1 ВЫБОР ТЕХНОЛОГИЙ РАЗРАБОТКИ	5
1.1 Система контроля версии Git.....	5
1.2 Сервис совместной разработки GitHub.....	5
1.3 Язык программирования Python	5
1.7 Фреймворк Django.....	6
2 USER STORY.....	7
3 ПОДГОТОВКА ПРОЕКТА	8
3.1 Создание корневой директории проекта	8
3.2 Создание веб-приложения Django.....	9
3.3 Создание репозитория в GitHub	10
4 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	11
4.1 Приложение аутентификации пользователей	11
4.1.1 Создание приложения и его подключение	11
4.1.2 Создание модели пользователя.....	13
4.1.3 Описание представлений.....	15
4.1.4 Описание тестов	18
4.2 Приложение галереи	18
4.2.1 Создание приложения и его подключение	18
4.2.2 Создание модели очереди	19
4.2.3 Описание представлений.....	20
4.2.4 Описание тестов	21

4.2.5 Commit изменений и push в репозиторий GitHub.....	21
ЗАКЛЮЧЕНИЕ	22

ВВЕДЕНИЕ

В рамках данного итогового проекта необходимо реализовать программное приложение с применением следующих технологий: системы контроля версий git, командной оболочки bash, а также менеджера пакетов для управления зависимостями и библиотеками.

В качестве темы итогового проекта по дисциплине «Технологии разработки программных приложений» была выбрана тема «Создание WEB-приложения для сохранения и отображения изображений». Это приложение будет направлено на возможность сохранения понравившихся изображений в отдельном ресурсе.

Проект включает в себя разработку как клиентской, так и серверной части, что позволит обеспечить полноценное взаимодействие пользователей с системой. Использование современных технологий, таких как git и bash, поможет в организации рабочего процесса и автоматизации задач.

Таким образом, данный проект не только позволит углубить знания в области разработки программных приложений, но и даст возможность применить на практике современные инструменты и подходы, что является важным аспектом в подготовке специалистов в области информационных технологий.

1 ВЫБОР ТЕХНОЛОГИЙ РАЗРАБОТКИ

1.1 Система контроля версии Git

В качестве системы контроля версии был выбран Git, так как это самое популярное решение для ОС Windows. Данная система поддерживается большинством хостингов IT-проектов для совместной разработки такими как GitHub, GitLab и другими.

Также при установке данного программного обеспечения устанавливается git bash, через который можно писать команды, доступные на ОС Linux, например, ls, mkdir, cat и другие.

1.2 Сервис совместной разработки GitHub

В качестве сервиса совместной разработки был выбран GitHub как один из популярных решений. Основным плюсом является возможность настроить CI/CD с помощью функции GitHub Actions, который позволяет проверять код перед принятием pull request или push'ем, а также деплоить проект на хостинг, где будет располагаться наш образ проекта. В данном случае будем использовать данную функцию для проверки кода на соблюдение тестам, а также на соблюдение стандарта PEP8 (для Python).

1.3 Язык программирования Python

Язык программирования Python является самым востребованным: он пользуется популярностью в таких сферах, где важна не скорость работы программы, а результат и понятный синтаксис. Не удивительно, что он используется в разработке серверной части WEB-приложений.

Более того, на Python написано большое количество хороших библиотек, которые предоставляют дополнительные возможности для обработки или получения данных, и хороших фреймворков для создания WEB-приложений, которые могут работать как обычные сервера сайтов, и как API для расширения количества устройств потребления информации.

1.4 Фреймворк Django

Для создания WEB-приложения сообщество Python-разработчиков предлагает нам большой выбор фреймворков на такую тематику: из них популярные это Flask, Django, FastAPI.

Среди вышеперечисленных фреймворков выделяется именно Django, поскольку в него вложена очень важная философия – разделение проекта на модули (или же здесь их называют приложениями). Также он предоставляет огромный инструментарий из классов, с помощью которых можно сократить код в разы по сравнению с тем же Flask.

Стоит упомянуть готовый интерфейс Django для администраторов сайта или API (если используется, например, Django REST framework).

2 USER STORY

Перед началом разработки программного приложения необходимо определиться с тем, что хочет пользователь от нашей Системы.

В Таблице 2.1 описан User Story в виде таблицы.

Таблица 2.1 – User Story

Как	Я хочу	Чтобы	Надо сделать
Незарегистрированный пользователь	Получить доступ к функционалу сайта	Сохранить понравившееся изображение	Страницу входа
Пользователь	Сохранить понравившееся изображение	Увидеть имеющиеся изображения и/или добавить новое	Страницу добавления изображения на сайт
Администратор	Иметь доступ к БД	Оперативно добавлять/удалять очереди/пользователей	Добавить модели в интерфейс администратора Django

3 ПОДГОТОВКА ПРОЕКТА

3.1 Создание корневой директории проекта

Для начала необходимо убедиться, что на ПК установлена версия Python 3.12: проверить можно с помощью команды `py --version`, который выведет версию Python по умолчанию (Рисунок 3.1.1).

```
Den@PCLite MINGW64 ~/Desktop/MySelf/STUDIES/work (main)
$ py --version
Python 3.13.2
```

Рисунок 3.1.1 – Вывод версии Python по умолчанию

В нашем случае проекта нет, поэтому необходимо открыть `bash` в нужной директории, где мы хотим видеть папку проекта и ввести команду `django-admin startproject i_watch work` (`i_watch` – название проекта, `work` – директория, в которой будет инициализирован проект) (Рисунок 3.1.2).

```
Den@PCLite MINGW64 ~/Desktop/MySelf/STUDIES (main)
$ django-admin startproject imagewatch work
```

Рисунок 3.1.2 – Инициализация проекта

После этого в появившейся папке проекта можно увидеть структуру, показанная на Рисунке 3.1.3.

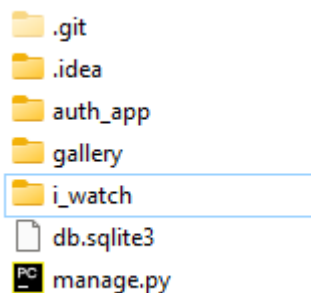


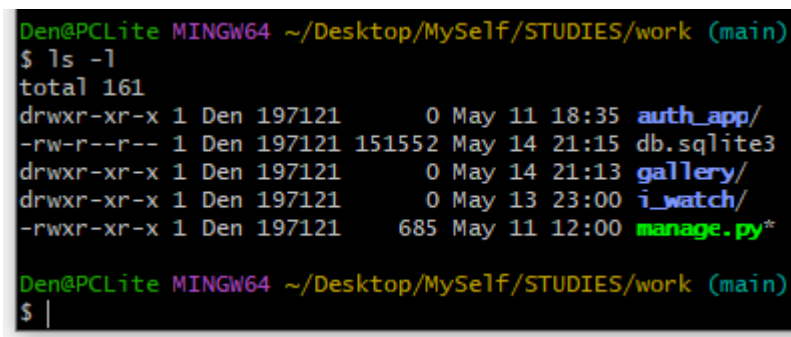
Рисунок 3.1.4 – Структура проекта после инициализации

Структура проекта после инициализации будет следующим:

- .git: папка для системы контроля версии Git. Устанавливается по умолчанию;
- manage.py: утилита, позволяющая взаимодействовать с проектом различными способами. Вы можете прочитать все подробности о manage.py в django-admin и manage.py.
- i_watch/: Каталог, который является фактическим пакетом Python для проекта. Его имя - это имя пакета Python, которое вам нужно будет использовать для импорта чего-либо из него (например, i_watch.urls).

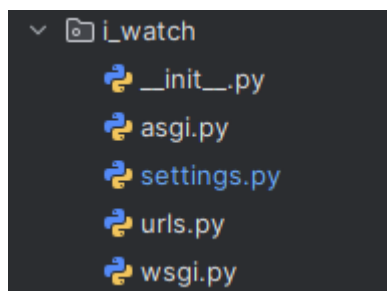
3.2 Создание веб-приложения Django

Для создания Django-проекта необходимо ввести команду: `django-admin startproject i_watch work`. После чего появится папка со структурой Django-проекта (Рисунок 3.3.1).



```
Den@PCLite MINGW64 ~/Desktop/MySelf/STUDIES/work (main)
$ ls -l
total 161
drwxr-xr-x 1 Den 197121      0 May 11 18:35 auth_app/
-rw-r--r-- 1 Den 197121 151552 May 14 21:15 db.sqlite3
drwxr-xr-x 1 Den 197121      0 May 14 21:13 gallery/
drwxr-xr-x 1 Den 197121      0 May 13 23:00 i_watch/
-rwxr-xr-x 1 Den 197121    685 May 11 12:00 manage.py*
```

Рисунок 3.3.1 – Структура проекта после создания Django-проекта
Давайте посмотрим на структуру папки fastqueue (Рисунок 3.3.3).



```
▼ i_watch
  __init__.py
  asgi.py
  settings.py
  urls.py
  wsgi.py
```

Рисунок 3.3.3 – Папка fastqueue

Рассмотрим каждый файл:

- __init__.py нужен в качестве инициализации директории для слаженной работы Python-приложения;

- в settings.py указываются настройки проекта;
- в urls.py указаны маршрутизаторы для обработки запросов со стороны клиента;
- wsgi.py требуется для настройки синхронного интерфейса;

3.3 Создание репозитория в GitHub

Публикация проекта в GitHub была реализована с помощью программы GitHub Desktop, которая позволяет быстро и без проблем создать облачный репозиторий.

В итоге репозиторий доступен по следующей ссылке: <https://github.com/Klarnold/work.git>. В дальнейшем именно в нем будут сохраняться все изменения в проекте.

4 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

4.1 Приложение аутентификации пользователей

Каждое приложение в фреймворке Django имеет следующие обязательные компоненты для его функционирования:

- модели (models) отвечают за взаимодействие с базой данных. Одна модель (класс, описанный в файле models.py) отвечает за одну таблицу объектов и описывает его атрибуты (столбцы) в реляционной базе данных;
- представления (views) отвечают за обработку запроса. С их помощью мы запрашиваем данные из таблицы и возвращаем пользователю заполненный шаблон HTML-страницы;
- маршрутизаторы (urls-файлы) отвечают за доставку запроса к нужному представлению.

4.1.1 Создание приложения и его подключение

В файле settings.py подключим наше приложение (Рисунок 4.1.1.1).

```
INSTALLED_APPS = [  
    'auth_app.apps.AuthAppConfig',  
    'gallery.apps.GalleryConfig',  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'rest_framework',  
]
```

Рисунок 4.1.1.1 – Подключение приложений

Теперь необходимо главному маршрутизатору сказать, что при некотором запросе перенаправлять его в приложение аутентификации. Для этого в `i_watch/urls.py` добавим запись на перенаправление в маршрутизатор приложения `auth_app` (Рисунок 4.1.1.2).

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("", include("gallery.urls")),
    path("auth_app/", include("auth_app.urls")),
    path("admin/", admin.site.urls),
]
```

Рисунок 4.1.1.2 – Код файла `i_watch/urls.py`

Для отладки пока будем использовать пустой путь к нашему приложению, то есть при запросе `/` нас перенаправит в это приложение.

Следующим шагом необходимо создать файл `auth_app/urls.py` и указать там пути к представлениям (Рисунок 4.1.1.3).

```
app_name="auth"
urlpatterns = [
    path('', views.index, name='index'),
    path("log_in", views.LogInView, name="log_in"),
    path("log_up", views.LogUpView, name="log_up"),
    path("log_out", views.log_out, name="log_out"),
    path("log_in/processing", views.log_in_processing_view, name="log_in_processing"),
    path("log_up/processing", views.log_up_processing_view, name="log_up_processing"),
]
```

Рисунок 4.1.1.3 – Код файла `auth_app/urls.py`

Здесь сперва мы указываем имя приложения. Это нужно для того, чтобы в дальнейшем различать названия представлений из разных приложений. Далее в массиве `urlpatterns` задаем маршруты и соответствующие им представления:

- пустой путь будет перенаправлять на представление `index`. В нем будет проверка на то, пользователь авторизован или нет;
- `log_in/` будет выдавать страницу с формой входа;

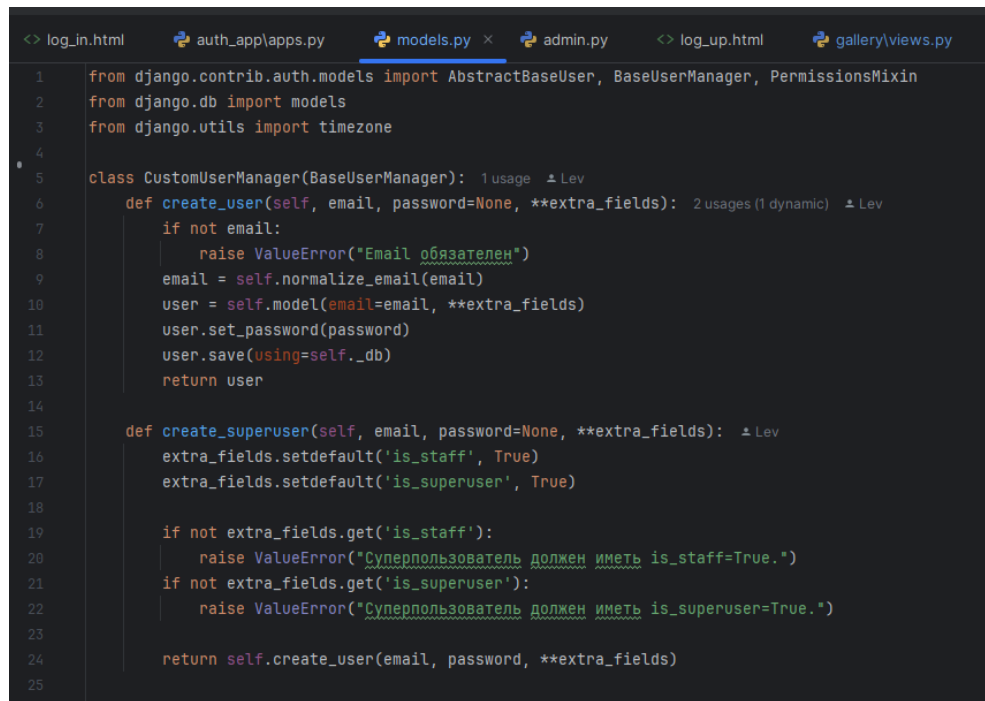
- `log_in/processing/` будет обрабатывать POST-запрос формы входа в аккаунт;
- `log_up/` будет выдавать страницу с формой регистрации;
- `log_up/processing/` будет обрабатывать POST-запрос формы регистрации;
- `log_out/` выполнит выход из приложения с последующим редиректом на страницу `log_in/`.

4.1.2 Создание модели пользователя

Django предоставляет два абстрактных класса для составления модели пользователя разработчиком: `AbstractUser` и `AbstractBaseUser`. Они отличаются тем, что при `AbstractBaseUser` разработчику необходимо описать все поля пользователя, в то время как `AbstractUser` предоставляет готовые поля такие как `username`, `email` и `password`.

Мы будем использовать второй класс, так как нам не нужен будет поле `username`. Вместо него будут поля `first_name` и `last_name`.

В файле `models.py` приложения `auth_app` необходимо помимо модели пользователя описать менеджер пользователей, унаследованный от `BaseUserManager`, так как мы используем `AbstractBaseUser`. В ней мы опишем функции создания обычного пользователя, а также создание суперпользователя – администратора сайта (Рисунок 4.1.2.1).



```

1 from django.contrib.auth.models import AbstractBaseUser, BaseUserManager, PermissionsMixin
2 from django.db import models
3 from django.utils import timezone
4
5 class CustomUserManager(BaseUserManager): 1 usage  ± Lev
6     def create_user(self, email, password=None, **extra_fields): 2 usages (1 dynamic)  ± Lev
7         if not email:
8             raise ValueError("Email обязателен")
9         email = self.normalize_email(email)
10        user = self.model(email=email, **extra_fields)
11        user.set_password(password)
12        user.save(using=self._db)
13        return user
14
15    def create_superuser(self, email, password=None, **extra_fields): ± Lev
16        extra_fields.setdefault('is_staff', True)
17        extra_fields.setdefault('is_superuser', True)
18
19        if not extra_fields.get('is_staff'):
20            raise ValueError("Суперпользователь должен иметь is_staff=True.")
21        if not extra_fields.get('is_superuser'):
22            raise ValueError("Суперпользователь должен иметь is_superuser=True.")
23
24        return self.create_user(email, password, **extra_fields)
25

```

Рисунок 4.1.2.1 – Описание менеджера пользователя

После описания менеджера пользователей необходимо описать саму модель пользователя в том же файле (Рисунок 4.1.2.2).



```

27 class CustomUser(AbstractBaseUser, PermissionsMixin): 8 usages  ± Lev *
28     email = models.EmailField(unique=True, verbose_name="Электронная почта")
29     first_name = models.CharField(max_length=30, verbose_name="Имя")
30     last_name = models.CharField(max_length=30, verbose_name="Фамилия")
31     is_active = models.BooleanField(default=True)
32     is_staff = models.BooleanField(default=False)
33     date_joined = models.DateTimeField(default=timezone.now)
34
35     objects = CustomUserManager()
36
37     USERNAME_FIELD = 'email'
38     REQUIRED_FIELDS = ['first_name', 'last_name']
39
40     def __str__(self): ± Lev
41         return self.email

```

Рисунок 4.1.2.2 – Модель пользователя

После создания пользователя необходимо его указать в settings.py как основную модель пользователя (Рисунок 4.1.2.3).

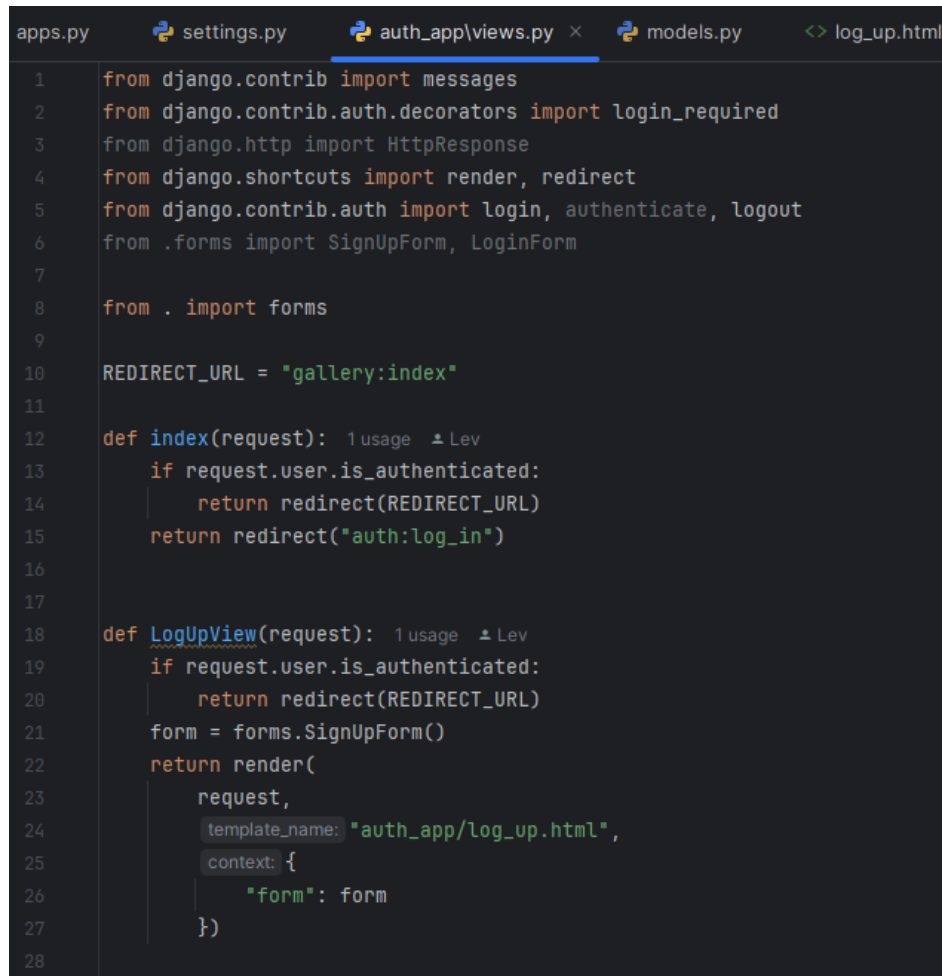
```
126
127     AUTH_USER_MODEL = 'auth_app.CustomUser'
128
```

Рисунок 4.1.2.3 – Указание модели пользователя как базовым

4.1.3 Описание представлений

После описания модели пользователя и настроек маршрутизаторов можно переходить к описанию представлений в файле views.py.

На Рисунках 4.1.3.1-4.1.3.3 представлен код представлений, которые были описаны в маршрутизаторе приложения.



```
apps.py  settings.py  auth_app\views.py  models.py  <> log_up.html
1  from django.contrib import messages
2  from django.contrib.auth.decorators import login_required
3  from django.http import HttpResponseRedirect
4  from django.shortcuts import render, redirect
5  from django.contrib.auth import login, authenticate, logout
6  from .forms import SignUpForm, LoginForm
7
8  from . import forms
9
10 REDIRECT_URL = "gallery:index"
11
12 def index(request): 1 usage 1 Lev
13     if request.user.is_authenticated:
14         return redirect(REDIRECT_URL)
15     return redirect("auth:log_in")
16
17
18 def LogUpView(request): 1 usage 1 Lev
19     if request.user.is_authenticated:
20         return redirect(REDIRECT_URL)
21     form = forms.SignUpForm()
22     return render(
23         request,
24         template_name="auth_app/log_up.html",
25         context={
26             "form": form
27         })
28
```

Рисунок 4.1.3.1 – Описание представлений index и LogUpView

```
apps.py settings.py auth_app\views.py x models.py log_up.html gallery\views.py auth_app\urls.py
29
30 def log_up_processing_view(request): 1 usage 1 Lev
31     if request.method == "POST":
32         form = forms.SignUpForm(request.POST)
33         if form.is_valid():
34             user = form.save(commit=False)
35             user.set_password(form.cleaned_data["password"])
36             user.save()
37             login(request, user)
38             return redirect(REDIRECT_URL)
39         else:
40             messages.error(request, message: "Пользователь с таким email уже существует или был введен простой пароль.")
41             return redirect("auth:log_up")
42     messages.error(request, message: "Что-то пошло не так.")
43     return redirect("auth:log_up")
44
45
46 def LogInView(request): 1 usage 1 Lev
47     if request.user.is_authenticated:
48         return redirect(REDIRECT_URL)
49     form = forms.SignInForm()
50     return render(
51         request,
52         template_name: "auth_app/log_in.html",
53         context: {
54             "form": form,
55         })
56
```

Рисунок 4.1.3.2 – Описание представлений log_up_processing_view и LogInView

```
58 def log_in_processing_view(request): 1 usage 1 Lev
59     if request.method == "POST":
60         form = forms.SignInForm(request.POST)
61         if form.is_valid():
62             user = form.cleaned_data["user"]
63             login(request, user)
64             return redirect(REDIRECT_URL)
65         messages.error(request, message: "Неправильный email или пароль.")
66         return redirect("auth:log_in")
67     messages.error(request, message: "Что-то пошло не так.")
68     return redirect("auth:log_in")
69
70
71 @login_required(login_url="auth:log_in") 1 usage 1 Lev
72 def log_out(request):
73     logout(request)
74     return redirect("gallery:index")
75
```

Рисунок 4.1.3.3 – Описание представлений log_in_processing_view и log_out

Также в представлениях используются классы форм, которые описаны в другом файле forms.py (Рисунки 4.1.3.4 – 4.1.3.5).

```
apps.py settings.py forms.py x auth_app\views.py models.py <> log
1 from django.contrib.auth.forms import UserCreationForm, AuthenticationForm
2 from django.contrib.auth.models import User
3 from django import forms
4
5
6 class LoginForm(AuthenticationForm): 1 usage 1 Lev
7     username = forms.CharField(label='Имя пользователя')
8     password = forms.CharField(label='Пароль', widget=forms.PasswordInput)
9
10 from django import forms
11 from django.contrib.auth import authenticate
12
13 from . import models
14
15
16 class SignInForm(forms.Form): 2 usages 1 Lev
17     email = forms.EmailField(label="Email")
18     password = forms.CharField(widget=forms.PasswordInput, label="Пароль")
19
20     def clean(self): 1 Lev
21         cleaned_data = super().clean()
22         email = cleaned_data.get('email')
23         password = cleaned_data.get('password')
24         user = authenticate(email=email, password=password)
25         if not user:
26             raise forms.ValidationError('Неправильный email или пароль.')
27         cleaned_data["user"] = user
28         return cleaned_data
29
30
```

Рисунок 4.1.3.4 – Часть 1 кода forms.py

```
31 class SignUpForm(forms.ModelForm): 3 usages 1 Lev
32     password = forms.CharField(widget=forms.PasswordInput, label="Пароль")
33     confirm_password = forms.CharField(widget=forms.PasswordInput, label="Подтверждение пароля")
34
35     class Meta: 1 Lev
36         model = models.CustomUser
37         fields = ['first_name', 'last_name', 'email', 'password', 'confirm_password']
38
39     def clean(self): 1 Lev
40         cleaned_data = super().clean()
41         password = cleaned_data.get('password')
42         confirm_password = cleaned_data.get('confirm_password')
43
44         if password and confirm_password and password != confirm_password:
45             raise forms.ValidationError("Пароли не совпадают.")
46
47         return cleaned_data
48
49     def clean_email(self): 1 Lev
50         email = self.cleaned_data.get('email')
51         if models.CustomUser.objects.filter(email=email).exists():
52             raise forms.ValidationError("Пользователь с таким адресом электронной почты уже существует.")
53         return email
54
```

Рисунок 4.1.3.5 – Часть 2 кода forms.py

4.1.4 Описание тестов

Одной из важнейших частей разработки программного продукта является написание UNIT-тестов для проверки кода на работу при вводе не только обычных данных, но и некорректных.

Важнейшим при написании UNIT-тестов является вопрос о списке тестов и их сценариях. В нашем приложении будут следующие UNIT-тесты:

- тест на вход на страницу аутентификации - `test_open_auth_index_page`.

Код файла `tests.py` можно увидеть в репозитории GitHub проекта по ссылке: <https://github.com/Klarnold/work/blob/main/gallery/tests.py>

4.2 Приложение галереи

Приложение изображений является основным с точки зрения бизнес-идеи. В ней будут описаны модели очередей и связки между очередью и клиентом, представления и маршрутизатор.

В качестве MVP-версии проекта будет использоваться синхронный интерфейс сайта, что будет означать отсутствие AJAX-запросов с клиентской стороны. Как следствие, мы создадим самую простую реализацию, но не самую надежную, поскольку при изменении состояния очереди необходимо динамически обновлять информацию о ней всем пользователям, которые открыли страницу с информацией об очереди.

4.2.1 Создание приложения и его подключение

Для создания нового приложения, как и в случае с предыдущим, необходимо ввести команду `python manage.py startapp gallery`, которая создаст новую директорию внутри папки проекта с названием `gallery`.

После чего необходимо упомянуть наше приложение в *settings.py* (Рисунок 4.2.1.1).

```
33  INSTALLED_APPS = [  
34      'auth_app.apps.AuthAppConfig',  
35      'gallery.apps.GalleryConfig',  
36      'django.contrib.admin',  
37      'django.contrib.auth',  
38      'django.contrib.contenttypes',  
39      'django.contrib.sessions',  
40      'django.contrib.messages',  
41      'django.contrib.staticfiles',  
42      'rest_framework',  
43  ]
```

Рисунок 4.2.1.1 – Упоминание Django-проекту о новом приложении

После чего поменяем файл *urls.py* так, чтобы при простом запросе нас автоматически перекидывало в созданное приложение (Рисунок 4.2.1.2).

```
els.py  gallery\views.py  auth_app\urls.py  tests.py  gallery\urls.py x  ⌵  ⋮  
1  from django.urls import path  
2  
3  from . import views  
4  from .views import *  
5  
6  app_name="gallery"  
7  urlpatterns = [  
8      path("", views.index, name="index"),  
9      path('api/image/', ImageListAPIView.as_view(), name='image-api'),  
10     path('api/image/create/', ImageCreateAPIView.as_view(), name='image-create'),  
11     path('api/add_image', views.add_image_view, name="add_image")  
12 ]
```

Рисунок 4.2.1.2 – Переопределение маршрутов в *urls.py*

4.2.2 Создание модели галереи

Главной моделью приложения будет являться модель очереди – *Image*. Данная модель должна иметь следующие атрибуты:

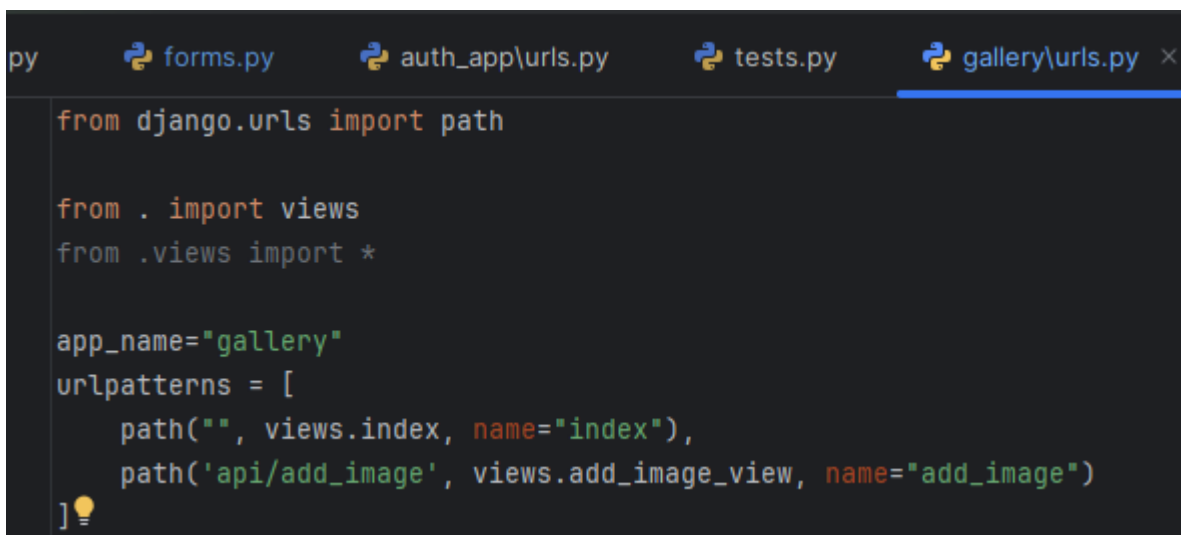
- user – внешний ключ модели пользователя для определения доступа к изображению;
- name – название очереди;
- urls_csv – это поле нужно для поиска изображения;

Код моделей Image и ImageSerializer можно посмотреть по ссылке: <https://github.com/Klarnold/work/blob/main/gallery/models.py> .

4.2.3 Описание представлений

Перед началом описания представлений приложения необходимо указать в файле gallery/urls.py пути, по которым можно будет в дальнейшем взаимодействовать со системой (Рисунок 4.2.3.1). Были указаны следующие пути:

- / - главная страница;
- api/add_image – ссылка на метод обработки добавления изображения;



```
py forms.py auth_app\urls.py tests.py gallery\urls.py x
from django.urls import path

from . import views
from .views import *

app_name="gallery"
urlpatterns = [
    path("", views.index, name="index"),
    path('api/add_image', views.add_image_view, name="add_image")
]
```

Рисунок 4.2.3.1 – Определение запросов в gallery/urls.py

Для каждого из этих путей были созданы отдельные представления, которые можно увидеть в файле gallery/views.py: <https://github.com/Klarnold/work/blob/main/gallery/views.py> .

4.2.4 Описание тестов

Тесты можно посмотреть в файле `test.py` (https://github.com/Klarnold/work/blob/main/auth_app/tests.py).

При запуске команды `python manage.py test` выводится сообщение об успешной проверке кода на тесты (Рисунок 4.2.4.1).

```
Den@PCLite MINGW64 ~/Desktop/MySelf/STUDIES/work (main)
$ python manage.py test
Found 1 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.
-----
Ran 1 test in 0.362s

OK
Destroying test database for alias 'default'...
Den@PCLite MINGW64 ~/Desktop/MySelf/STUDIES/work (main)
$ |
```

Рисунок 4.2.4.1 – Результат выполнения тестов

4.2.5 Commit изменений и push в репозиторий GitHub

Изменения были зафиксированы в ветке (Рисунок 4.2.5.1).

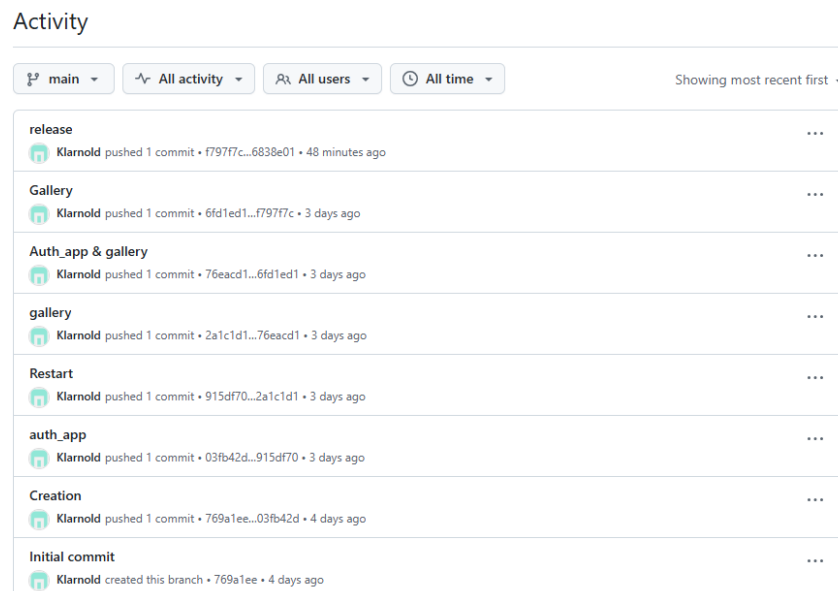


Рисунок 4.2.5.1 – Отображение коммитов в ветке main

ЗАКЛЮЧЕНИЕ

В ходе выполнения Итогового проекта по дисциплине «Технологии разработки программных приложений» были закреплены на практике знания о таких технологиях как Git, облачный сервис IT-проектов GitHub, а также были закреплены умения работы с ними.

Репозиторий проекта можно посмотреть по ссылке: <https://github.com/Klarnold/work.git> . Репозиторий имеет лицензию MIT, который позволяет использовать, копировать, изменять, публиковать, распространять, сублицензировать и продавать этот код без каких-либо ограничений с сохранением авторского права за автором.